EXERCISE - 1

Write a Program in C/ C++ for error detecting code using CRC-CCITT (16 bit)

Objective:

To detect errors if any in the transmitted data using CRC-CCITT 16 bit polynomial.

The aim of an error detection technique is to enable the receiver of a message transmitted through noisy channel to determine whether the message has been corrupted. To do this, the transmitter constructs a value called a checksum that is the function of message and appends it's to message. The receiver can then use the same function to calculate the checksum of the received message and compare it with the appended checksum to see if the message was correctly received.

IMPLEMENTATION

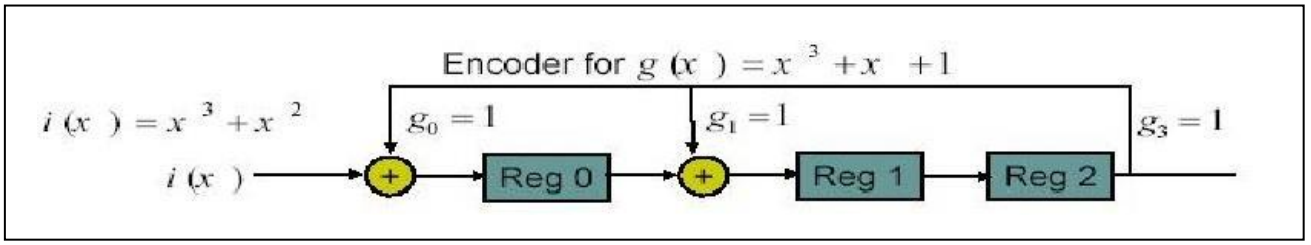

Figure 1: Euclidean division algorithm

The message is represented by a information polynomial i(x). i(x) is store as a bit pattern of k length in an integer array. The k information bits are represented by k-1 degree polynomial

i(x) = i(k-1)x(k-1) + i(k-2)x(k-2) +..... +i1x+i0

A polynomial code is specified by its generating polynomial g(x). If we assume that we are dealing with a code in which codewords have n bits of which k are information bits and n-k are check bits. The generator polynomial for such a code has degree n-k and has the form

g(x) = x(n-k) + g(n-k-1)x(n-k-1) +.....+g1x+1

The generator polynomial chosen is CCITT-16 (x16+x12+x5+1).

Encoding Procedure at Sender

Steps

1) Multiply i(x) by x(n-k) by putting zeros in n-k low order positions

2) Divide x(n-k)i(x) by g(x) to get r(x). Use Euclidean division algorithm with a feedback shift register as shown in above figure
   x(n-k)i(x) = g(x) q(x) + r(x) where q(x) is quotient and r(x) is remainder
3) Add remainder r(x) to x(n-k)i(x) by putting check bits in n-k lower order positions

4) Based on randomness, the message can be transmitted with error or without error.

5) For transmission with error, introduce an error at random position to the message x(n-k)i(x) and display the position of the error.

6) Transmitted codeword is b(x) = x(n-k)i(x) + r(x)

Decoding Procedure at the Receiver

Steps

1) The received message b(x) is divided by g(x) using Euclidean division algorithm

2) If the remainder is 0 then there is no error in the transmission else Error in the transmission.

EXERCISE - 2

Write a Program in C/ C++ for hamming code generation for error detection/correction

Objective:

To Detect and Correct Single bit errors using Hamming Code.

Hamming Code is used to detect and correct single bit error. The key to Hamming Code is the use of extra parity bits. Hamming code consists of k information bits and n-k check bits, where n is the total number of bits in the codeword. Parity bits are placed in positions having power of 2.

IMPLEMENTATION (7,4) HAMMING CODE:

- The Hamming Code for 4-bits of data uses 3 redundant bits.

Hamming Code for 4 data bits

d4	d3	d2	d1

is

d4	d3	d2	*r3*	d1	*r2*	*r1*	
b7	b6	b5	b4	b3	b2		b1...(position of bits in code)

where r1 r2 and r3 are redundant bits.

PROCEDURE FOR DETERMINING REDUNDANT BITS:

The ability of detecting and correcting errors of Hamming Code comes with the cost of redundant bits. These 3-bits are used to take care of all the 8 different possible states of transmitted 7-bits.

- The r-bits are determined using following equations:

d1 = r1 + r2 |
d2 = r1 + r3 | modulo-2 arithmetic
d3 = r2 + r3 |
d4 = r1 + r2 + r3 |

These equations are further solved to calculate r-bits:

r1 = d1 + d2 + d4
r2 = d1 + d3 + d4
r3 = d2 + d3 + d4

PROCEDURE FOR ERROR DETECTION AND CORRECTION:

- On the receiver side 3 position bits(p1-p3) are calculated:

  p1 = r1 + d1 + d2 + d4
  p2 = r2 + d1 + d3 + d4
  p3 = r3 + d2 + d3 + d4

- These values indicate one of the eight possible states of received code:

p3 p2 p1	state	Action for correction
0 0 0	No error	--
0 0 1	Error in b1 bit	Flip b1
0 1 0	Error in b2 bit	Flip b2
0 1 1	Error in b3 bit	Flip b3
1 0 0	Error in b4 bit	Flip b4
1 0 1	Error in b5 bit	Flip b5
1 1 0	Error in b6 bit	Flip b6
1 1 1	Error in b7 bit	Flip b7

And d4-d1 bits after this operation are the actual transferred data bits.

Steps to execute the program

At Sender: Code Formation:

1. Get 4-bit(d4-d1) input
2. Determine r-bits(r3-r1) using above equations
3. Form the 7-bit Code by placing r-bits and d-bits in appropriate positions.
4. Send the Code to Receiver

At receiver: Error Detection and Correction:

1. Determine position bits(p3-p1) using above equations
2. Use the Table to find the position of error (if present) and to take corrective action.

EXERCISE - 3

Write a Program in C/ C++ for client server communication using TCP or IP sockets to make client send the name of the file and server to send back the contents of the requested file if present.

Objective: To Use TCP/IP sockets and write a client server program in which the client sends the file name in request message and the server sends back the contents of the requested file if present.

Sockets is a method for communication between a client program and a server program in a network. A socket is defined as "the endpoint in a connection." Sockets are created and used with a set of programming requests or "function calls" sometimes called the sockets application programming interface (API). The most common sockets API is the Berkeley UNIX C interface for sockets. Sockets can also be used for communication between processes within the same computer.

Creating a Socket

The socket system call is given below:

```
s = socket(domain, type, protocol);
```

The domain is either AF_UNIX or AF_INET. An AF_UNIX socket can only be used for interprocess communications on a single system, while an AF_INET socket can be used for communications between systems.

The type specifies the characteristics of communication on the socket. SOCK_STREAM creates a socket that will reliably deliver bytes in-order, but does not respect messages boundaries; SOCK_DGRAM creates a socket that does respect message boundaries, but does not guarantee to deliver data reliably, uniquely or in order. A SOCK_STREAM socket corresponds to TCP; a SOCK_DGRAM socket corresponds to UDP

The protocol selects a protocol. Ordinarily this is 0, allowing the call to select a protocol. This is almost always the right thing to do, though in some special cases you may want to select the protocol yourself. Remember that this refers to the underlying network protocol: such well-known protocols as http, ftp, and ssh are all built on top of tcp, so tcp is the right choice for any of those protocols.

For example:

```
s = socket(AF_INET, SOCK_STREAM, 0);
```

will create a socket that will use TCP to communicate. The return value (s) is a file descriptor for the socket.

Binding a Socket

At this point created a socket, but name is not given to it. To give a name to the socket by using the bind system call:

```
bind(s, name, namelen);
```

This call gives the socket a name. For an Internet socket, the name is a struct defined as

```
struct sockaddr_in {
   sa_family_t    sin_family; /* address family: AF_INET */
   u_int16_t      sin_port;   /* port in network byte order */
   struct in_addr  sin_addr;  /* internet address */
};

/* Internet address. */
struct in_addr {
   u_int32_t      s_addr;     /* address in network byte order */
};
```

sin_family is always AF_INET; sin_port is the port number, and sin_addr is the IP address. Port numbers below 1024 are reserved - that means only processes with an effective user id of 0 (ie the root) can bind to those ports.

Listening to the Socket

Once the socket has been created and bound, the daemon needs to indicate that it is ready to listen to it. It does this with the listen system call, as in

```
listen(s, 5);
```

The main thing this does is to set a limit on how many would-be clients can be queued up trying to connect to the socket (the limit in this example is 5). If the limit is exceeded the clients don't actually get refused, instead their connection requests get dumped on the floor. Eventually they will end up retrying.

Accepting Connections

The server is able to accept connections by calling accept:

```
newsock = accept(s, (struct sockaddr *)&from, &fromlen);
```

For this call, s is, as you'd expect the socket that was returned oh so long ago by the socket call. The accept() call blocks until the client connects to the socket.

Connecting to the daemon

A client connects to the socket using the connect call. First it creates a socket using the socket call, then it connects it to the daemon's socket using connect:

```
connect(s, (struct sockaddr *)&server, sizeof(server));
```

This call returns *a new socket*. This means the daemon can communicate with the client using the newly created (and unnamed) socket, while continuing to listen on the old one.

At this point, the server normally forks a child process to handle the client, and goes back to its accept loop.

The child doing the communication can either use standard read and write calls, or it can use send and recv. These calls work like read and write, except that you can also pass flags allowing for some options.

Sending Data

There are a variety of functions that may be used to send outgoing messages.

write() may be used in exactly the same way as it is used to write to files. This call may only be used with SOCK_STREAM type sockets.

```
#include<sys/types.h>
#include<sys/socket.h>
int write(int fd, char *msg, int len);
```

fd is the socket descriptor. **msg** specifies the buffer holding the text of the message, and **len** specifies the length of the message.

write() is similar to write() except that it writes from a set of buffers. This is called a gather write. This call may only be used with SOCK_STREAM type sockets.

send() may be used in the same way as write(). The prototype is

```
#include<sys/types.h>
#include<sys/socket.h>
int send(int fd, char *msg, int len, int flags);
```

fd is the socket descriptor. **msg** specifies the buffer holding the text of the message, and **len** specifies the length of the message. **flags** may be formed by ORing MSG_OOB and MSG_DONTROUTE. The latter is only useful for debugging. This call may only be used with SOCK_STREAM type sockets.

Receiving Data

There are a variety of functions that may be used to receive incoming messages.

read() may be used in the exactly the same way as for reading from files. There are some complications with non-blocking reads. This call may only be used with SOCK_STREAM type sockets.

#include <sys/types.h>
#include <sys/socket.h>

int read(int fd, char *buff, int len)

fd is the socket descriptor. **buff** is the address of a buffer area. **len** is the size of the buffer.

readv() may be used in the same way as read() to read into several separate buffers. This is called a scatter read. This call may only be used with SOCK_STREAM type sockets.

recv() may be used in the same way as read(). The prototype is

#include <sys/types.h>
#include <sys/socket.h>

int recv(int s, char *buff, int len, int flags)

buff is the address of a buffer area. **len** is the size of the buffer. **flags** is formed by ORing MSG_OOB and MSG_PEEK allowing receipt of out of band data and allowing peeking at the incoming data. This call may only be used with SOCK_STREAM type sockets.

Steps to execute the exercise

- Client side
    1) sfd = Create a socket with the socket(….) system call
    2) Connect the socket to the address of the server using the connect(sfd, …) system call. The IP address of the server machine and port number of the server service need to be provided.
    3) Read file name from standard input by n = read(stdio, buffer, sizeof(buffer))
    4) Write file name to the socket using write (sfd, buffer, n)
    5) Read file contents from the socket by m = read(sfd, buffer1, sizeof(buffer1))
    6) Display file contents to standard output by write(stdio, buffer1, m)
    7) Go to step 5 if m>0
    8) Close socket by close (sfd)
- Server side

    1) sfd = Create a socket with the socket(…) system call

2) Bind the socket to an address using the bind (sfd, …) system call. If not sure of machine IP address, keep the structure member s_addr to INADDR_ANY. Assign a port number between 3000 and 5000 to sin_port.
3) Listen for connections with the listen (sfd, …) system call
4) sfd = Accept a connection with the accept (sfd, …) system call. This call typically blocks until a client connects with the server.
5) Read the filename from the socket by n = read(sfd, buffer, sizeof(buffer))
6) Open the file by fd = open(buffer)
7) Read the contents of the file by m = read(fd, buffer1, sizeof(buffer1))
8) Write the file content to socket by write(sfd, buffer1, m)
9) Go to step 7 if m>0
10) Close(sfd)

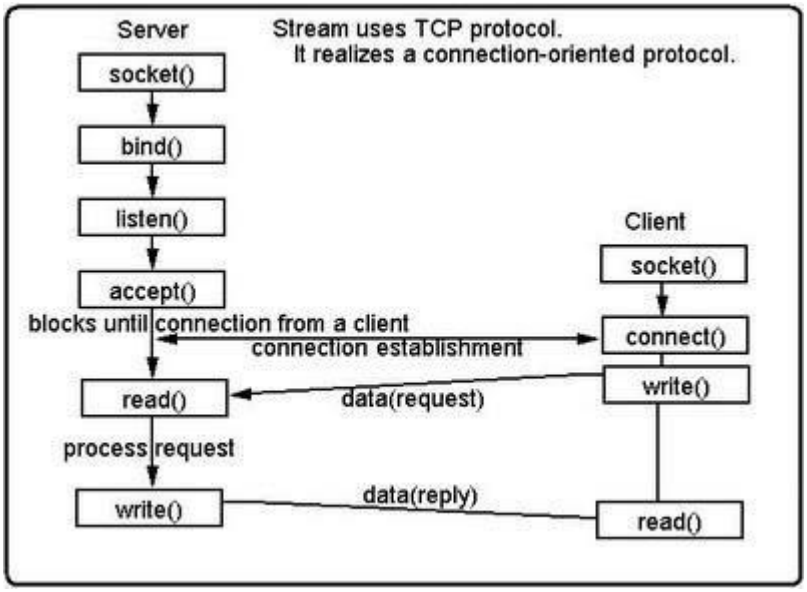

Figure 2: TCP sockets

EXERCISE - 4

Write a Program in C/ C++ for traffic Policing using Leaky Bucket algorithm

Objective: To implement traffic policing using Leaky Bucket

Leaky bucket is a traffic-shaping algorithm implemented by the sender in a network. The algorithm provides for a steady data transmission rate irrespective of the variation in data generation rate.

The program for implementation leaky bucket algorithm can be viewed in two parts

1. Generating the data in variable bursts.
2. Data transmission at constant rate.

Leaky bucket can be implemented using a circular queue to hold data before it can be transmitted. The size of the circular queue is the size of the bucket size. The data generated from the process is inserted into the queue. When the queue is full, the generated data is discarded. Data get removed from the queue in FIFO fashion.

By using interval timers, generate data in random bursts and store them to bucket. Using timers data from the bucket can be transmitted at a constant rate periodically. *Setitimer* API is use to set interval timers to the process to carry out the above functions. The API is defined in sys/time.h header file.

Setitimer API prototype:

int setitimer (int which, const struct itimerval *val, struct itimerval *old);

Setitimer allows up to three timers to be set for a process, and offers a resolution time in microseconds. Expiry of the timer generates signal determined by the 'which' argument in the API that can be either TIMER_REAL (SIGALRM) or TIMER_VIRTUAL (SIGVTALRM) or TIMER_PROF (SIGPROF).

We can use the API to set two timers, for generating data and for transmitting data. When the timers go off respective actions associated with the signal handler are called to perform the operations.

For generating random burst of data we use the random () function when the timer associated with data generation goes off.

Socket programming or FIFO files can be use to transmit data to the receiver.

Steps to execute the program:

Main function

1. Establish connection from Sender to Receiver using socket programming or FIFO files.
2. Associate Send data procedure to handle SIGALRM signal
3. Associate Generate_data procedure to handle SIGVTARLM signal.
4. Set the timer values for ITIMER_REAL timer and ITIMER_VIRTUAL timer in microseconds.
5. Set up the two timers using setitimer API.
6. WHILE TRUE DO
    a. Wait for timers to go off periodically

Generate_data function:

1. Generate a packet.
2. IF bucket (queue) is full DO

    Discard the packet generated.

ELSE

    Insert the generated data into the bucket (queue).

Send_data function:

1. IF bucket (queue) not empty

    a. Remove data form bucket (queue).
    b. Send the removed data to the receiver.

ELSE

    Display ‘Bucket empty’.

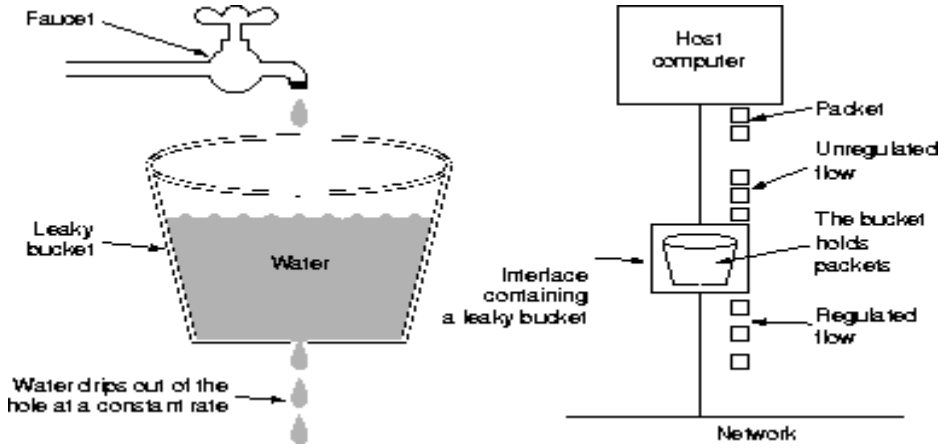

Figure 3: Leaky Bucket